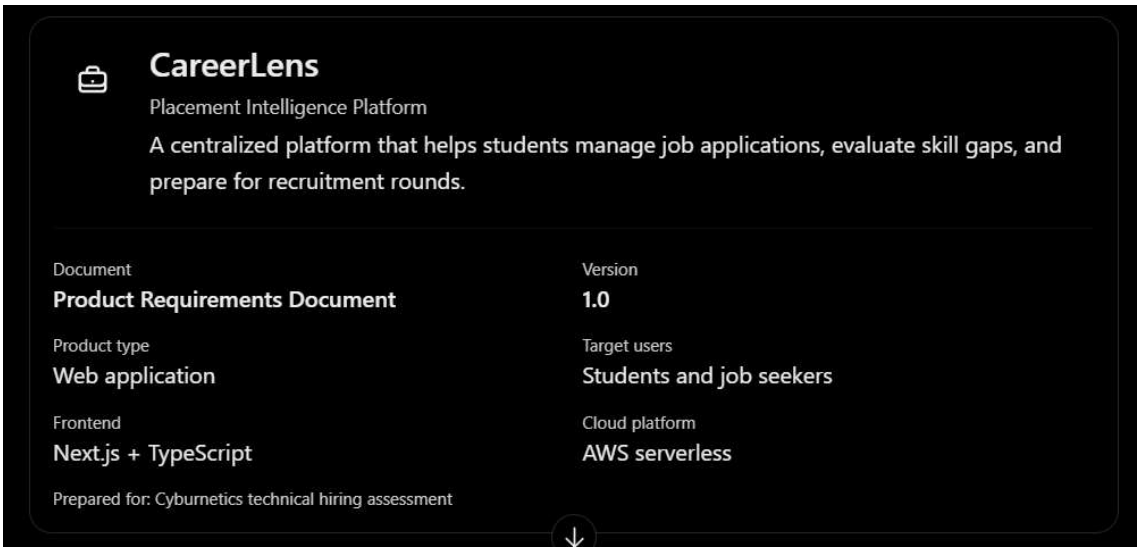


1. Executive summary

CareerLens is a web-based placement management system designed to help students organize their job search in one place.

Students often apply to multiple companies through different job portals, maintain separate resumes, prepare for different skill requirements, and track interviews manually. CareerLens addresses this problem by providing a unified dashboard for application tracking, job description analysis, skill-gap identification, and recruitment preparation.

The platform will allow users to record job applications, save job descriptions, compare their skills against job requirements, monitor recruitment stages, and review their placement progress through analytics.

The application will use a serverless AWS backend to support secure data storage, file uploads, and API-driven business logic.

2. Problem statement

Students preparing for campus placements and off-campus recruitment frequently face the following challenges:

- **Difficulty tracking applications across multiple companies.**
- **Missing application deadlines and interview dates.**
- **Limited understanding of the skills required for specific roles.**
- **Difficulty identifying gaps between existing skills and job descriptions.**
- **Disorganized resume versions and recruitment documents.**
- **No centralized view of interview progress and application outcomes.**

Problem to solve: Build a centralized, reliable, and easy-to-use platform that helps students manage the recruitment lifecycle and make data-informed preparation decisions.

3. Product objectives

CareerLens will aim to achieve the following objectives.

1. **Centralize job applications and recruitment-stage tracking.**
2. **Enable users to compare their existing skills with job requirements.**
3. **Help students identify missing skills and prioritize preparation.**
4. **Store resumes and relevant job documents securely.**
5. **Provide a dashboard showing application status, upcoming interviews, and recruitment statistics.**
6. **Deliver a responsive, performant web application using the required AWS architecture.**

Success criteria

Metric	Initial target
Application creation	Complete successfully with valid input
Skill comparison	Correctly identify matched and missing listed skills
Data persistence	Records remain available after refresh and re-login
File uploads	Valid resume files upload and can be securely accessed
Responsive UI	Usable on desktop, tablet, and mobile
API reliability	Expected responses and meaningful errors for tested cases
Test coverage	All critical user workflows covered by automated or documented tests

These are proposed project acceptance targets, not existing product measurements.

4. Target users

Primary user: Student or job seeker

- **Maintains a professional profile and skill inventory.**
- **Tracks applications to companies.**
- **Compares skills with job descriptions.**
- **Records interviews, deadlines, and outcomes.**

Secondary user: Platform administrator

For the initial assessment, administrative capabilities will be limited to essential account support and platform oversight if time permits. A full college placement-office portal is outside the MVP scope.

5. Product scope

5.1 Features included in the MVP

User authentication and profile

Secure registration and login, profile details, and editable skill inventory.

Application tracker

Create, view, edit, delete, search, filter, and sort job applications.

Skill-gap analyzer

Extract or enter required skills and compare them with a user's declared skills using a transparent matching algorithm.

Recruitment timeline

Track stages such as Applied, Assessment, Interview, Offer, Rejected, and Withdrawn.

Resume management

Upload PDF resumes to S3, associate versions with applications, and securely download them.

Analytics dashboard

Display application counts, stage distribution, interview schedules, and skill-gap summaries.

5.2 Features excluded from the MVP

To keep the project achievable and reliable, the initial release will not include:

- **Automatic job scraping from external portals.**
- **Direct job applications submitted to third-party websites.**
- **AI-generated hiring predictions or guaranteed placement scores.**
- **Real-time integration with company applicant tracking systems.**
- **College-wide placement administration and multi-tenant management.**
- **Paid subscription billing.**

These features can be considered after the core application is stable.

6. Functional requirements

The following identifiers will be used later in the development and testing documents.

ID	Requirement	Priority
FR-01	Users can register, log in, and log out securely.	Must
FR-02	Users can view and update their profile and skills.	Must
FR-03	Users can create and manage job applications.	Must
FR-04	Users can assign a recruitment status to each application.	Must
FR-05	Users can store a job description for each relevant role.	Must
FR-06	The system compares required skills with the user's skill inventory.	Must
FR-07	Users can upload and manage PDF resume versions.	Must

FR-08	The dashboard shows application and recruitment statistics.	Must
FR-09	Users can search, filter, and sort applications.	Must
FR-10	Users can record interview dates, notes, and outcomes.	Should
FR-11	The system provides reminders for upcoming deadlines.	Should
FR-12	Users can export application information as a CSV file.	Could

Priority definitions: Must = required for the MVP; Should = valuable if time allows; Could = optional enhancement.

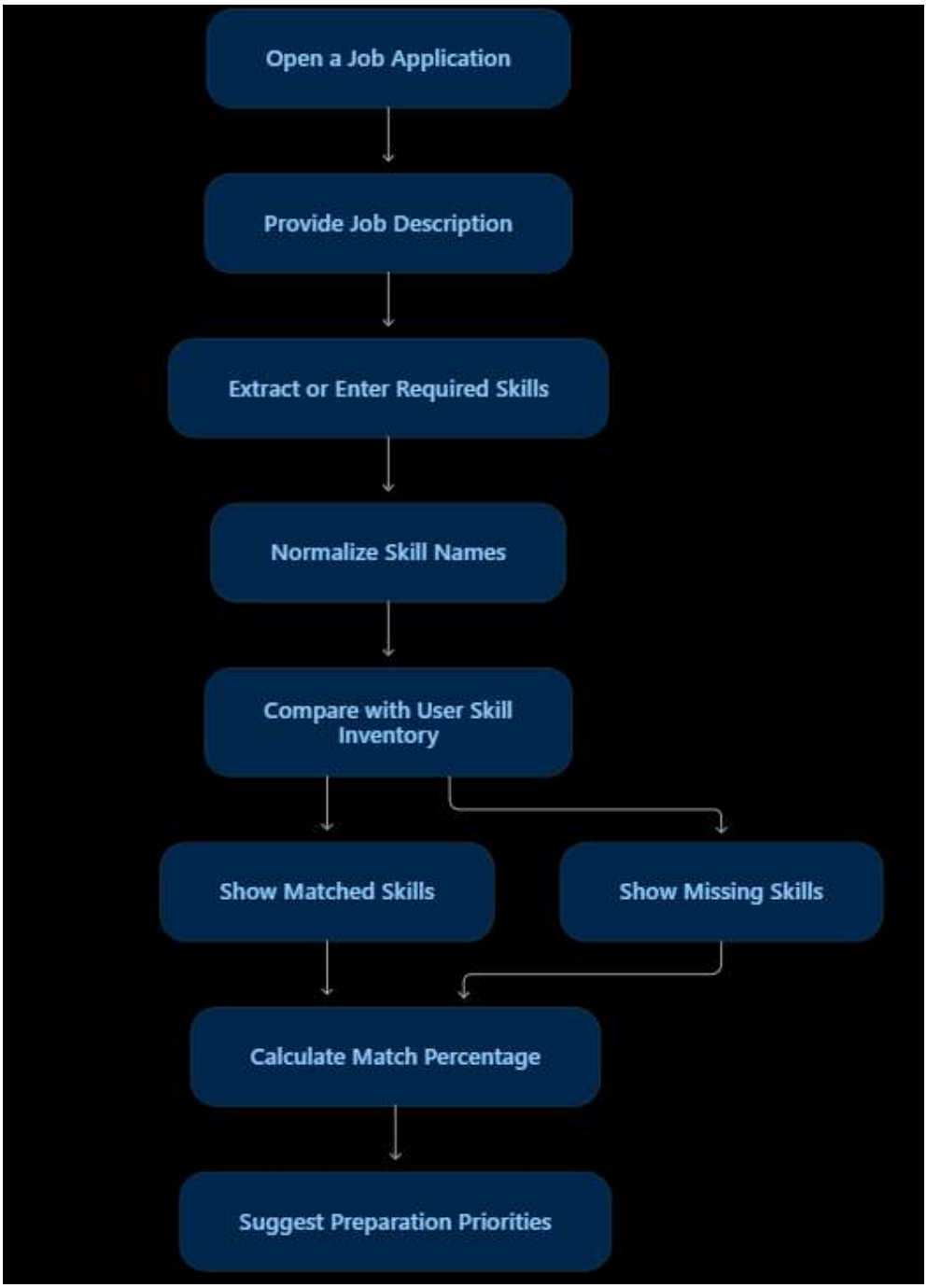
7. Core user workflows

Workflow A — Manage a job application



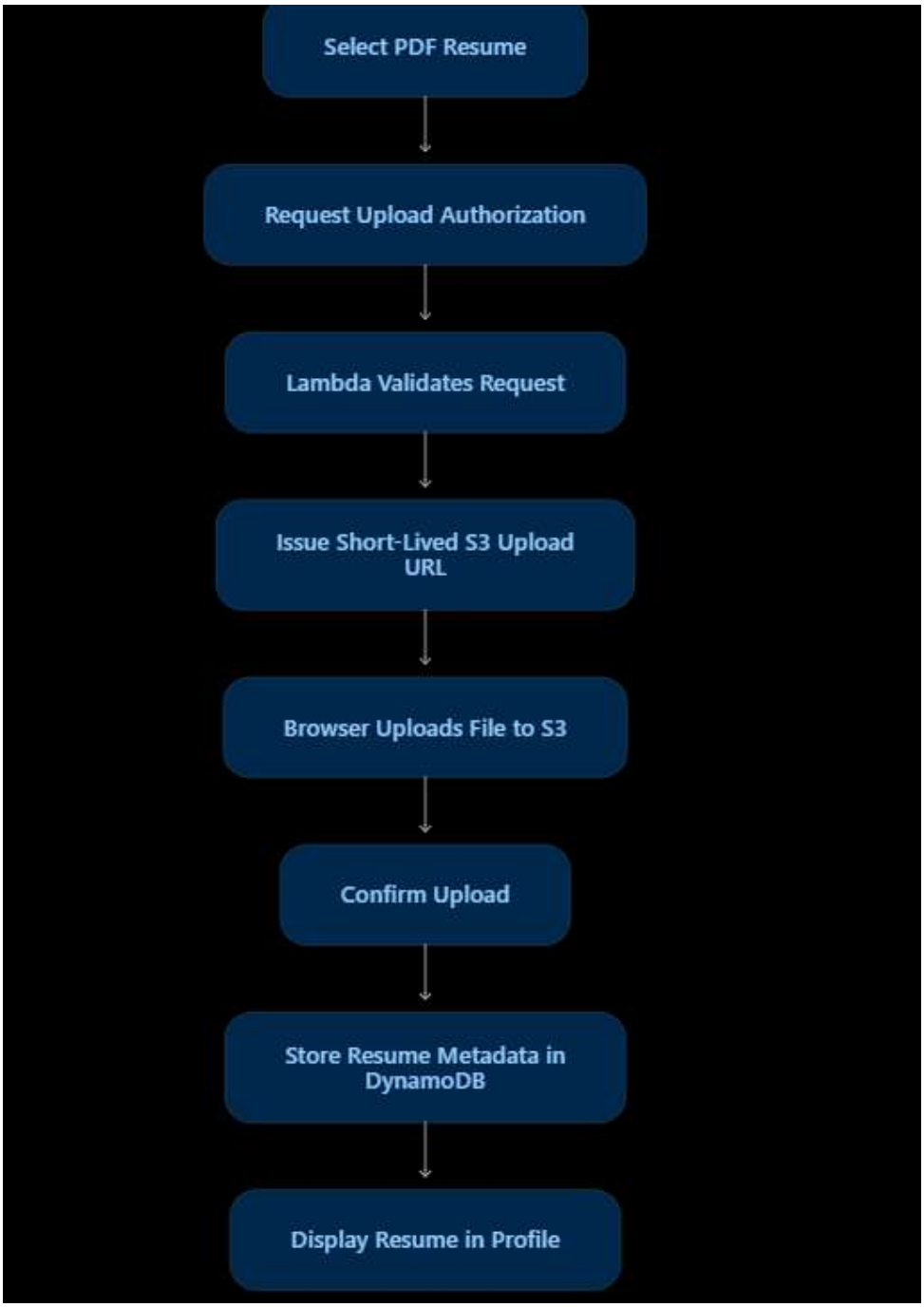
Expected result: the application is saved, appears in the tracker, and remains available when the user returns.

Workflow B — Analyze skill gaps



The initial implementation should use a deterministic, explainable matching algorithm. For example, if 6 of 10 distinct required skills match, the skill match percentage is 60%. This is a preparation aid, not a measure of employability or hiring probability.

Workflow C — Upload a resume



The bucket should remain private. The server must validate the user's authorization and the uploaded object's metadata; a file extension alone is not sufficient proof of file type.

8. Non-functional requirements

Performance

- Use paginated API responses for application lists.
- Avoid unnecessary database scans.
- Load dashboard summaries efficiently.
- Optimize frontend bundles, images, and rendering.
- Use appropriate caching only where it does not expose private user data.
- Measure API latency and frontend loading performance during testing.

Security and privacy

- Require authenticated access to private user data.
- Enforce ownership checks on every application, resume, and profile operation.
- Store secrets outside source code.
- Use least-privilege IAM permissions.
- Keep S3 buckets private and use short-lived presigned URLs.
- Validate all input on the backend.
- Avoid logging passwords, access tokens, or resume contents.
- Provide a clear account-data deletion approach.

Reliability

- Return consistent API error responses.
- Handle expired sessions and failed uploads.
- Prevent unauthorized changes to another user's records.
- Log operational errors using CloudWatch.
- Keep database writes and application state consistent.

Usability

- Provide clear navigation and helpful empty states.
- Use readable forms with inline validation.
- Support mobile, tablet, and desktop layouts.
- Show loading indicators, success messages, and actionable errors.

- Use accessible labels and keyboard-friendly interactions.

9. Proposed technical constraints

Component	Technology
Frontend	Next.js, TypeScript
UI styling	Tailwind CSS
Authentication	Amazon Cognito
API layer	Amazon API Gateway
Backend logic	AWS Lambda
Database	Amazon DynamoDB
Resume storage	Amazon S3
Hosting	AWS Amplify Hosting
Monitoring	Amazon CloudWatch

Testing	Vitest or Jest, React Testing Library, API integration tests

The application must not depend on EC2. The selected hosting and deployment approach must be compatible with the chosen Next.js features; if using Amplify, verify its support for the specific framework configuration before deployment.

10. High-level data requirements

The platform will maintain the following logical entities. Their physical DynamoDB key design will be specified in Document 3.

Entity	Important fields
User	user ID, name, email, skills, profile metadata
Job Application	application ID, user ID, company, role, status, dates
Job Description	application ID, description text, required skills
Interview	interview ID, application ID, date, stage, notes
Resume	resume ID, user ID, S3 object key, version, upload date

Skill Analysis	application ID, matched skills, missing skills, percentage

All user-owned records must be scoped to the authenticated user's identity. The final schema should support the required queries without relying on inefficient full-table scans.

11. Proposed application pages

Dashboard

Application summary, upcoming interviews, skill-gap overview.

Applications

Searchable table, filters, status badges, and application details.

Skill Analyzer

Required skills, matched skills, missing skills, and preparation checklist.

Interview Calendar

Upcoming interview dates, stages, notes, and outcomes.

Resume Vault

Private resume uploads, versions, and application associations.

Profile and Settings

Personal information, skill inventory, and account controls.

12. Acceptance criteria

The MVP is considered ready for submission when all of the following are satisfied:

- A user can register, authenticate, and access their own dashboard.
- A user can create, edit, view, and delete a job application.
- Application statuses and interview dates persist correctly.
- Skill comparison returns correct matched and missing skill lists.
- Resume uploads and downloads work through authorized S3 access.
- Dashboard metrics reflect the user's stored applications.
- Unauthorized access to another user's records is rejected.
- Invalid inputs and failed requests produce understandable errors.
- The interface works on common desktop and mobile screen sizes.
- The application is deployed to AWS without EC2.
- The repository includes setup instructions and a deployment overview.

13. Risks and mitigations

Risk	Mitigation
Scope grows too large	Prioritize the Must requirements
AWS setup consumes the deadline	Deploy a small end-to-end flow early
Incorrect skill matching	Use normalized skill names and deterministic rules

Unauthorized data access	Test ownership checks on every private endpoint
DynamoDB queries become inefficient	Design access patterns before finalizing keys
S3 upload failures	Add size/type validation and retry-friendly error handling
Hosting incompatibility	Verify framework support before choosing deployment settings

14. Development milestones

A practical sequence is:

1. Finalize MVP requirements and wireframes.
2. Create the Next.js application and shared UI components.
3. Configure authentication and AWS resources.
4. Implement application CRUD and DynamoDB persistence.
5. Implement skill analysis and dashboard metrics.
6. Add secure S3 resume uploads.
7. Run unit, integration, authorization, and responsive UI tests.
8. Deploy, verify the production URL, and prepare the README and demo.

For a short hiring deadline, the first priority should be a complete end-to-end flow rather than optional reminders, advanced analytics, or AI features.

15. Future enhancements

After the MVP is stable, CareerLens could add:

- Resume parsing and automatic skill extraction.
- AI-assisted job description analysis with user-reviewed results.
- Calendar integrations and reminder notifications.
- Job listing imports from authorized sources.
- Exportable placement reports.
- Optional placement-office features with carefully designed access controls.

These enhancements are not required for the initial assessment.